FIRST FAULT IDENTIFICATION:

CODE:

```
#include <reg51.h>
#include <intrins.h>  // for nop()

// ---------- Inputs ----------
sbit overheat = P3^4;     // SW1 : active LOW
sbit overload = P3^3;     // SW2 : active LOW

// ---------- LEDs ----------
sbit led_overheat = P1^0;
sbit led_overload = P1^1;
sbit led_timer    = P1^2;

// ---------- 7-segment digit enables ----------
sbit DIG0 = P2^0;   // left-most digit
sbit DIG1 = P2^1;
sbit DIG2 = P2^2;
sbit DIG3 = P2^3;

// ---------- Segment codes ----------
#define S_CODE 0x6D
#define A_CODE 0x77
#define F_CODE 0x71
#define E_CODE 0x79
#define H_CODE 0x76
```

```
#define T_CODE 0x78
#define L_CODE 0x38
#define O_CODE 0x3F
#define D_CODE 0x5E
#define DASH  0x40

// ---------- Words ----------
unsigned char code_SAFE[4] = {S_CODE, A_CODE, F_CODE, E_CODE};
unsigned char code_HEAT[4] = {H_CODE, E_CODE, A_CODE, T_CODE};
unsigned char code_LOAD[4] = {L_CODE, O_CODE, A_CODE, D_CODE};

// ---------- Display buffer ----------
unsigned char display_buf[4] = {S_CODE, A_CODE, F_CODE, E_CODE};
unsigned char digit_index = 0;

// ---------- Timer Counters ----------
unsigned int ms_count = 0;

// ---------- Software RTC Simulation ----------
unsigned char sec = 0, min = 46, hour = 22;  // Start at 22:46
unsigned char colon_flag = 0;

// ---------- UART ----------
void uart_init(void) {
   TMOD |= 0x20;
   TH1 = 0xFD;
```

```
    SCON = 0x50;
    TR1 = 1;
}

void uart_tx(char c) {
    SBUF = c;
    while(!TI);
    TI = 0;
}

void uart_send_string(char *s) {
    while(*s) uart_tx(*s++);
}

void uart_print2digit(unsigned char val) {
    uart_tx('0' + (val/10));
    uart_tx('0' + (val%10));
}

// ---------- 7-segment numbers ----------
unsigned char seg_num[10] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};

// ---------- Timer0 ISR ----------
void timer0_ISR(void) interrupt 1 {
    TH0 = 0xFC;
    TL0 = 0x66;
```

```
// Multiplex display
P2 &= ~0x0F;
P0 = display_buf[digit_index];

switch(digit_index){
   case 0: DIG0=1; break;
   case 1: DIG1=1; break;
   case 2: DIG2=1; break;
   case 3: DIG3=1; break;
}
digit_index = (digit_index+1)&0x03;

ms_count++;
if(ms_count >= 1000){
   ms_count = 0;
   colon_flag = !colon_flag;
   led_timer = ~led_timer;

   // --- Software RTC increment ---
   sec++;
   if(sec >= 60){
      sec = 0;
      min++;
      if(min >= 60){
         min = 0;
```

```
            hour++;
            if(hour >= 24) hour = 0;
          }
        }
    }
}

// ---------- Init ----------
void timer0_init(void){
    TMOD |= 0x01;
    TH0 = 0xFC;
    TL0 = 0x66;
    ET0 = 1;
    TR0 = 1;
    EA  = 1;
}

// ---------- Display Update ----------
void update_display(void){
    static bit last_overheat = 1;
    static bit last_overload = 1;

    if(overload == 0){  // Overload pressed
        display_buf[0]=code_LOAD[0];
        display_buf[1]=code_LOAD[1];
        display_buf[2]=code_LOAD[2];
```

```
    display_buf[3]=code_LOAD[3];
    led_overload = 1;
    led_overheat = 0;

    if(last_overload == 1){ // Just pressed
        uart_send_string("OVERLOAD ACTIVE\r\n");
    }
    last_overload = 0;
}
else if(overheat == 0){  // Overheat pressed
    display_buf[0]=code_HEAT[0];
    display_buf[1]=code_HEAT[1];
    display_buf[2]=code_HEAT[2];
    display_buf[3]=code_HEAT[3];
    led_overheat = 1;
    led_overload = 0;

    if(last_overheat == 1){ // Just pressed
        uart_send_string("OVERHEAT ACTIVE\r\n");
    }
    last_overheat = 0;
}
else{  // SAFE state - show time
    display_buf[0]=seg_num[hour/10];
    display_buf[1]=seg_num[hour%10];
    display_buf[2]=seg_num[min/10];
```

```
      display_buf[3]=seg_num[min%10];

      if(colon_flag) display_buf[1] |= DASH;

      led_overheat = 0;
      led_overload = 0;

      // When switches are released ? print SAFE with time
      if(last_overheat == 0){
        uart_send_string("SAFE STATE (After Overheat): ");
        uart_print2digit(hour); uart_tx(':'); uart_print2digit(min);
        uart_send_string("\r\n");
      }
      if(last_overload == 0){
        uart_send_string("SAFE STATE (After Overload): ");
        uart_print2digit(hour); uart_tx(':'); uart_print2digit(min);
        uart_send_string("\r\n");
      }

      last_overheat = 1;
      last_overload = 1;
    }
}

// ---------- Main ----------
void main(void){
```

```
    P0=0x00; P1=0x00; P2=0x00; P3=0xFF; // inputs high
  uart_init();
  timer0_init();

  uart_send_string("System Started... Initial Time = 22:46\r\n");

  while(1){
    update_display();
  }
}
```

Working:

- P3.3 is connected to overload and P3.4 is connected to overheat.
- When overload is pressed, system will show “overload active” and when it is released, it will display “safe state with current time”.
- When overheat is pressed, system will show “overheat active” and when it is released, it will display “safe state with current time”.
- When overload is pressed first followed by overheat, uart will show “overload active” alone and not “overheat active” since overload is high priority and when it is released, it will display “safe state with current time”.
- When overheat is pressed first followed by overload, uart will show “overheat active” and then “overload active” since overheat is low priority and when it is released, it will display “safe state with current time”.
- That’s how interrupt, timer, uart and RTC works in this project whenever overload and overheat occurs.